

ARYABHATTA KNOWLEDGE UNIVERSITY, PATNA

BCA-204 — PROBLEM SOLVING TECHNIQUE & PROGRAMMING IN C

Paper Code: 303204

Exam: 23 September 2026 | 10:00 AM – 1:00 PM | Full Marks: 60

ANSWER-SHEET SCOPE

This printable sheet contains the 10 long-answer questions identified in the uploaded BCA-204 exam guide, with the stated PYQ years and exam-copy answers. The uploaded guide describes 7 units and the 60-mark, 3-hour AKU pattern.

QUESTIONS COVERED

1. Explain all types of operators in C with examples and operator precedence. — 12 Marks — PYQ: 2015, 2016, 2019
2. Write and explain the structure of a C program. Also explain identifiers, keywords and data types with sizes. — 12 Marks — PYQ: Every year (as stated in the uploaded exam guide)
3. What do you mean by call by value and call by reference? Write a program to swap two numbers using both methods. — 12 Marks — PYQ: 2015, 2016, 2019, 2021
4. What are advantages and disadvantages of recursive function? Write a program to print Fibonacci series using recursion. Also write program for factorial. — 12 Marks — PYQ: 2015, 2016, 2017
5. What is an array? Explain 1D and 2D arrays with memory layout, initialization, and programs. Also explain passing arrays to functions. — 12 Marks — PYQ: 2014, 2015, 2016, 2019
6. Give the advantage of array using pointers. Write a program to access elements of an array by using pointers. Also write program to find length of string using pointer. — 12 Marks — PYQ: 2015, 2016
7. What is structure? What is union? Explain their differences. Write programs using both. Also explain self-referential structure. — 12 Marks — PYQ: 2015, 2016, 2017, 2018, 2022
8. What is a pointer in C? Explain pointer declaration, initialization, operations, pointer arithmetic, pointer with arrays, and pointer with structures. Give examples. — 12 Marks — PYQ: 2018, 2019, 2022
9. Explain the C Preprocessor. What are its advantages? Explain all directives: #define, #include, #ifdef, #ifndef, conditional compilation with programs. — 12 Marks — PYQ: 2016, 2017, 2021, 2025
10. What is a file? Why are files needed? How to open and close a file? Explain all file modes and all file I/O functions with a complete program. — 12 Marks — PYQ: 2015, 2016, 2019, 2022, 2025

AKU WRITING ORDER

Introduction → Definition → numbered points → syntax/example → program/diagram where needed → conclusion. Underline key terms and label diagrams clearly.

UNIT 1 — FUNDAMENTALS OF C

12 Marks

PYQ years: 2015, 2016, 2019

Question: Explain all types of operators in C with examples and operator precedence.

Introduction: An operator is a symbol that tells the computer to perform a specific mathematical or logical operation on operands. C provides arithmetic, relational, logical, assignment, unary, conditional, bitwise and comma operators.

1. **Arithmetic:** +, -, *, /, %. Example: $10/3 = 3$ and $10\%3 = 1$.
2. **Relational:** ==, !=, >, <, >=, <= . They return 1 (true) or 0 (false).
3. **Logical:** && (AND), || (OR), ! (NOT). Used to combine conditions.
4. **Assignment:** =, +=, -=, *=, /=, %=. Example: $a += 5$ means $a = a + 5$.
5. **Unary:** ++, --, !, ~ and sizeof(). Pre-increment changes first; post-increment uses the value first.
6. **Conditional/Ternary:** condition ? value_if_true : value_if_false. Example: $\text{max} = (a > b) ? a : b$;
7. **Bitwise:** &, |, ^, ~, <<, >>. Example: $5 \& 3 = 1$.
8. **Comma:** Evaluates expressions left to right and gives the rightmost value. Example: $c = (a = 2, b = 4, a + b)$; gives 6.

Diagram / Flow:

Operator precedence (high $\rightarrow$ low):

() $\rightarrow$ unary $\rightarrow$ * / % $\rightarrow$ + - $\rightarrow$ relational $\rightarrow$ == != $\rightarrow$ && $\rightarrow$ || $\rightarrow$?: $\rightarrow$ assignment

Conclusion

Operator precedence decides which operator is evaluated first. Parentheses can be used to change the normal order. A useful order is: arithmetic $\rightarrow$ relational $\rightarrow$ logical $\rightarrow$ assignment.

Memory Trick

Remember: A-R-L-A = Arithmetic, Relational, Logical, Assignment.

12 Marks

PYQ years: Every year (as stated in the uploaded exam guide)

Question: Write and explain the structure of a C program. Also explain identifiers, keywords and data types with sizes.

Introduction: C is a structured, procedural, middle-level programming language developed by Dennis Ritchie at Bell Labs in 1972. A C program is organized into preprocessor directives, declarations, main() and executable statements.

1. **Preprocessor directives:** #include and #define are processed before compilation.
2. **Global declarations:** Variables/functions declared outside main() can have wider scope.
3. **main() function:** It is the entry point of program execution.
4. **Local declarations:** Variables declared inside a function/block are local to that scope.
5. **Statements:** They contain the actual program logic.
6. **return statement:** return 0; normally indicates successful termination.

Diagram / Flow:

```
#include <stdio.h>
Global declarations
```

↓

```
main()
```

↓

```
Local declarations  $\rightarrow$  Statements  $\rightarrow$  return 0
```

Conclusion

Identifiers name program elements, keywords are reserved words, and data types specify the kind of data stored. A clear program structure makes C programs easier to compile, understand and maintain.

Memory Trick

I-K-D-S: Identifier rules $\rightarrow$ Keywords $\rightarrow$ Data type $\rightarrow$ Structure.

UNIT 2 — FUNCTIONS

12 Marks

PYQ years: 2015, 2016, 2019, 2021

Question: What do you mean by call by value and call by reference? Write a program to swap two numbers using both methods.

Introduction: Arguments can be passed to a function by value or by reference. In C, ordinary parameter passing is by value; reference-like modification is achieved by passing addresses through pointers.

Call by Value: A copy of the actual argument is passed. Changes inside the function do not change the original variables.

Call by Reference: The address of the actual argument is passed using pointers. Changes through dereferencing affect the original variables.

Important operators: & gives an address; * accesses the value stored at an address.

Diagram / Flow:

```
main: x, y
    |       |
    | values | addresses (&x, &y)
    |       |
    ↓       ↓
value fn   pointer fn
(no original change) (original changes)
```

Program:

```
#include <stdio.h>

void swap_val(int a, int b) {
    int t=a; a=b; b=t;
}
void swap_ref(int *a, int *b) {
    int t=*a; *a=*b; *b=t;
}
int main() {
    int x=5, y=10;
    swap_val(x,y);
    printf("By Value: %d %d\n",x,y);
    swap_ref(&x,&y);
    printf("By Ref: %d %d\n",x,y);
    return 0;
}

/* Output:
By Value: 5 10
By Ref: 10 5
*/
```

Conclusion

Call by value protects the original values, while pointer-based reference passing can modify them. For swapping original variables, pass their addresses and dereference inside the function.

Memory Trick

Value = COPY; Reference = ADDRESS.

12 Marks

PYQ years: 2015, 2016, 2017

Question: What are advantages and disadvantages of recursive function? Write a program to print Fibonacci series using recursion. Also write program for factorial.

Introduction: Recursion is a technique in which a function calls itself. A recursive solution must contain a base case and a recursive call that moves toward that base case.

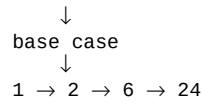
Advantages: Shorter code; natural for factorial, Fibonacci, tree traversal and divide-and-conquer problems.

Disadvantages: Function-call overhead, extra stack memory, possible stack overflow and harder debugging.

Base case: Stops recursion. Without it, the function may call itself indefinitely.

Diagram / Flow:

```
fact(4)
↓
4×fact(3) → 3×fact(2) → 2×fact(1)
```

**Program:**

```
#include <stdio.h>

long long factorial(int n) {
    if(n==0 || n==1) return 1;
    return n*factorial(n-1);
}

int fib(int n) {
    if(n<=1) return n;
    return fib(n-1)+fib(n-2);
}

int main() {
    int n=5;
    printf("Factorial=%lld\n", factorial(n));
    printf("Fibonacci: ");
    for(int i=0; i<n; i++) printf("%d ", fib(i));
    return 0;
}

/* Output:
Factorial=120
Fibonacci: 0 1 1 2 3
*/
```

Conclusion

Recursion is useful when a problem naturally breaks into smaller versions of itself. Always provide a correct base case and a recursive step.

Memory Trick

BASE + CALL = recursion.

UNIT 3 — ARRAYS & STRINGS

12 Marks

PYQ years: 2014, 2015, 2016, 2019

Question: What is an array? Explain 1D and 2D arrays with memory layout, initialization, and programs. Also explain passing arrays to functions.

Introduction: An array stores multiple values of the same data type under one name. Elements are stored in contiguous memory and are accessed using an index beginning at 0.

1D declaration: `int a[5];` Initialization: `int a[5]={10,20,30,40,50};`

2D declaration: `int m[2][3];` Example: `{{1,2,3},{4,5,6}}.`

Access: `a[i]` accesses a one-dimensional element; `m[i][j]` accesses a two-dimensional element.

Passing to function: Pass the array name and its size, e.g. `void printArr(int a[], int n).`

Key relation: `a[i]` is equivalent to `*(a+i).`

Diagram / Flow:

1D: [10] [20] [30] [40] [50]
 0 1 2 3 4

2D: col0 col1 col2
row0 1 2 3
row1 4 5 6

Address (row-major): `Base + (i*columns+j)*sizeof(type)`

Program:

```
#include <stdio.h>
void printArr(int a[], int n) {
    for(int i=0;i<n;i++) printf("%d ",a[i]);
}
int smallest(int a[], int n) {
    int min=a[0];
    for(int i=1;i<n;i++)
        if(a[i]<min) min=a[i];
    return min;
}
int main() {
    int a[5]={30,10,50,20,40};
    printArr(a,5);
    printf("\nSmallest=%d",smallest(a,5));
    return 0;
}
/* Output:
30 10 50 20 40
Smallest=10
*/
```

Conclusion

Arrays provide simple indexed storage and fast direct access. They have fixed size and store elements of one data type. Passing the array name gives the function access to the original array.

Memory Trick

Index starts at 0; last index = size-1.

12 Marks

PYQ years: 2015, 2016

Question: Give the advantage of array using pointers. Write a program to access elements of an array by using pointers. Also write program to find length of string using pointer.

Introduction: The array name represents the address of its first element in expressions. Pointers can therefore traverse arrays and strings efficiently using pointer arithmetic.

Array-pointer relation: `a[i] = *(a+i),` and `&a[i] = a+i.`

Advantages: Efficient traversal, easy function passing, direct memory access and convenient string traversal.

Pointer arithmetic: `p+1` moves by `sizeof` of the pointed data type, not necessarily one byte.

String length: Move a char pointer until `'\0'` and count the characters.

Diagram / Flow:

a → [10][20][30][40][50]
 ↑
 p
p+i → element i

String: H e l l o \0
 ↑ stop

Program:

```
#include <stdio.h>

void accessArr(void) {
    int a[5]={10,20,30,40,50};
    int *p=a;
    for(int i=0;i<5;i++)
        printf("a[%d]=%d\n",i,*(p+i));
}

int strLen(char *s) {
    int len=0;
    while(*s!='\0') { len++; s++; }
    return len;
}

int main() {
    accessArr();
    printf("Length=%d",strLen("Hello"));
    return 0;
}

/* Output:
a[0]=10 ... a[4]=50
Length=5
*/
```

Conclusion

Pointers make array traversal compact and efficient. The fundamental identity to remember is $a[i] = *(a+i)$.

Memory Trick

$INDEX = * (BASE + i)$.

UNIT 4 — STRUCTURES, UNIONS & BITWISE

12 Marks

PYQ years: 2015, 2016, 2017, 2018, 2022

Question: What is structure? What is union? Explain their differences. Write programs using both. Also explain self-referential structure.

Introduction: Structure and union are user-defined data types used to group different kinds of data. A structure gives separate storage to members, while a union shares the same storage.

Structure: Groups different data types; each member has separate memory; all members can contain valid values together.

Union: All members share the same memory; only one member's value is reliably active at a time; size is based on the largest member.

Self-referential structure: A structure containing a pointer to the same structure type, used for linked lists and trees.

Access: Use . with a structure variable and -> with a pointer to structure.

Diagram / Flow:

STRUCTURE: [roll][name][marks] → separate memory
UNION: [shared memory]

Self-reference:
[data|next] → [data|next] → NULL

Program:

```
#include <stdio.h>

struct Student { int roll; char name[20]; float marks; };
union Data { int i; float f; char c; };
struct Node { int data; struct Node *next; };

int main() {
    struct Student s={101,"Zoha",85.5};
    printf("%d %s %.1f\n",s.roll,s.name,s.marks);

    union Data d;
    d.i=10;
    printf("i=%d\n",d.i);
    d.f=3.14;
    printf("f=%.2f\n",d.f);
    return 0;
}
/* Output:
101 Zoha 85.5
i=10
f=3.14
*/
```

Conclusion

Use a structure when all fields are needed at the same time. Use a union when memory saving is important and only one member is needed at a time. Self-referential structures are fundamental to dynamic data structures.

Memory Trick

STRUCT = Separate; UNION = Shared.

12 Marks

PYQ years: 2018, 2019, 2022

Question: What is a pointer in C? Explain pointer declaration, initialization, operations, pointer arithmetic, pointer with arrays, and pointer with structures. Give examples.

Introduction: A pointer is a variable that stores the memory address of another variable. Pointers are central to arrays, functions, structures and direct memory manipulation in C.

Declaration: data_type *ptr; Example: int *p;

Initialization: int a=10; int *p=&a; Here p stores the address of a and *p gives 10.

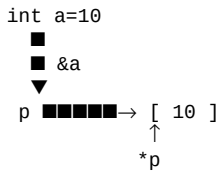
Pointer operations: Dereference with *, obtain address with &, and use valid pointer arithmetic such as p++, p--, p+n and p-n.

Arrays: p=a points to the first element; *(p+i) accesses element i.

Functions: Passing addresses allows a function to modify original variables.

Structures: A structure pointer accesses members using ->.

Diagram / Flow:



Program:

```
#include <stdio.h>
struct Point { int x; int y; };

int main() {
    int a[5]={10,20,30,40,50};
    int *p=a;
    for(int i=0;i<5;i++)
        printf("%d ",*(p+i));

    struct Point pt={3,4};
    struct Point *pp=&pt;
    printf("\nx=%d y=%d",pp->x,pp->y);
    return 0;
}
/* Output:
10 20 30 40 50
x=3 y=4
*/
```

Conclusion

Pointers store addresses. Remember & for address-of and * for dereference. Pointer arithmetic depends on the pointed data type size. Always initialize pointers and avoid using invalid or dangling addresses.

Memory Trick

& = address; * = value at address.

UNIT 6 — C PREPROCESSOR

12 Marks

PYQ years: 2016, 2017, 2021, 2025

Question: Explain the C Preprocessor. What are its advantages? Explain all directives: #define, #include, #ifdef, #ifndef, conditional compilation with programs.

Introduction: The C preprocessor is a program that processes source code before compilation. It handles directives beginning with # and produces modified source code for the compiler.

#define: Creates symbolic constants and function-like macros. Example: #define PI 3.14159 and #define SQUARE(x) ((x)*(x)).

#include: Includes header files. Angle brackets are normally used for system headers; quotes are used for user headers.

#ifdef / #ifndef: Compile code when a macro is defined or not defined.

Other directives: #undef removes a macro; #if/#else/#elif/#endif provide conditional compilation; #error issues a compile-time error; #pragma is compiler-specific.

Advantages: Macro substitution, header inclusion, conditional compilation and platform/configuration-specific code.

Diagram / Flow:

Source.c → PREPROCESSOR → Modified source → COMPILER → Object → LINKER → Program

Program:

```
#include <stdio.h>
#define PI 3.14159
#define AREA(r) (PI*(r)*(r))
#define DEBUG

int main() {
    float r=5.0;
    printf("Area=%.2f\n", AREA(r));
#ifdef DEBUG
    printf("Debug mode ON\n");
#endif
    return 0;
}
/* Output:
Area=78.54
Debug mode ON
*/
```

Conclusion

The preprocessor works before the compiler. It performs text substitution and conditional inclusion rather than normal run-time computation. Macros are fast but do not provide function-like type checking.

Memory Trick

D-I-C: Define, Include, Condition.

12 Marks

PYQ years: 2015, 2016, 2019, 2022, 2025

Question: What is a file? Why are files needed? How to open and close a file? Explain all file modes and all file I/O functions with a complete program.

Introduction: A file is a collection of related data stored permanently on secondary storage. File handling lets a C program save, retrieve and process data after the program ends.

Why files are needed: Permanent storage, large data storage, data sharing, logging and repeated input/output.

FILE pointer: FILE *fp; is used by the C library to manage an open file.

Opening and closing: fp=fopen("name","mode"); check fp==NULL; perform operations; fclose(fp);

Modes: r = read; w = write/create and erase existing contents; a = append; r+ = read/write existing; w+ = read/write with truncation; a+ = read/append.

Text I/O: fprintf/fscanf for formatted I/O; fputc/fgetc for characters; fputs/fgets for strings.

Binary I/O: fread and fwrite read/write blocks of binary data.

File positioning: fseek moves the file position; ftell returns the current position; rewind returns to the beginning.

Diagram / Flow:

```
FILE *fp
↓
fopen(name, mode)
↓
check NULL
↓
read / write / seek
↓
fclose(fp)
```

Program:

```
#include <stdio.h>

int main() {
    FILE *fp;
    int roll; char name[20]; float marks;

    fp=fopen("student.txt", "w");
    if(fp==NULL) return 1;
    scanf("%d %s %f",&roll,name,&marks);
    fprintf(fp,"%d %s %.2f\n",roll,name,marks);
    fclose(fp);

    fp=fopen("student.txt", "r");
    if(fp==NULL) return 1;
    while(fscanf(fp,"%d %s %f",&roll,name,&marks)!=EOF)
        printf("Roll:%d Name:%s Marks:%.2f\n",roll,name,marks);
    fclose(fp);
    return 0;
}
/* Example output:
Roll:101 Name:Zoha Marks:85.50
*/
```

Conclusion

The standard file sequence is FILE pointer → fopen() → check NULL → perform I/O → fclose(). Remember that w can erase an existing file, while a appends data. fseek, ftell and rewind support file positioning.

Memory Trick

OPEN → CHECK → I/O → CLOSE.

LAST PAGE — QUICK REVISION TABLE

Q	Unit	Core question	PYQ years
1	U1	Explain all types of operators in C with examples and operator precedence.	2015, 2016, 2019
2	U1	Write and explain the structure of a C program. Also explain identifiers, keywor...	Every year (as stated in the uploaded exam guide)
3	U2	What do you mean by call by value and call by reference? Write a program to swap...	2015, 2016, 2019, 2021
4	U2	What are advantages and disadvantages of recursive function? Write a program to ...	2015, 2016, 2017
5	U3	What is an array? Explain 1D and 2D arrays with memory layout, initialization, a...	2014, 2015, 2016, 2019
6	U3	Give the advantage of array using pointers. Write a program to access elements o...	2015, 2016
7	U4	What is structure? What is union? Explain their differences. Write programs usin...	2015, 2016, 2017, 2018, 2022
8	U5	What is a pointer in C? Explain pointer declaration, initialization, operations,...	2018, 2019, 2022
9	U6	Explain the C Preprocessor. What are its advantages? Explain all directives: #de...	2016, 2017, 2021, 2025
10	U7	What is a file? Why are files needed? How to open and close a file? Explain all ...	2015, 2016, 2019, 2022, 2025

EXAM-WRITING CHECKLIST

1. Start every long answer with a direct definition/introduction.
2. Use numbered headings and underline important terms.
3. Write syntax before the program when a syntax-based question is asked.
4. For programs: include #include, main(), declarations, logic, return 0; and a short output.
5. Draw a small labelled diagram when it clarifies memory, flow or file handling.
6. For comparison questions, use a table.
7. End with a 1–2 line conclusion.
8. Do not write unrelated theory when time is limited.